Assignment 3 - Hybrid Auto-Scaling System (Local VM → Monitoring → Auto Migration to GCP)

Student Name: Aryan Baranwal
Roll No.: M25CSE035
Course: Virtualization and Cloud Computing

1. Objective

To implement a **hybrid cloud auto-scaling mechanism** that:

- Monitors CPU & RAM usage on a local Ubuntu VM
- Detects threshold breach
- Automatically creates a VM instance on Google Cloud
- Deploys a Flask application on GCP
- Cleans up cloud resources after execution

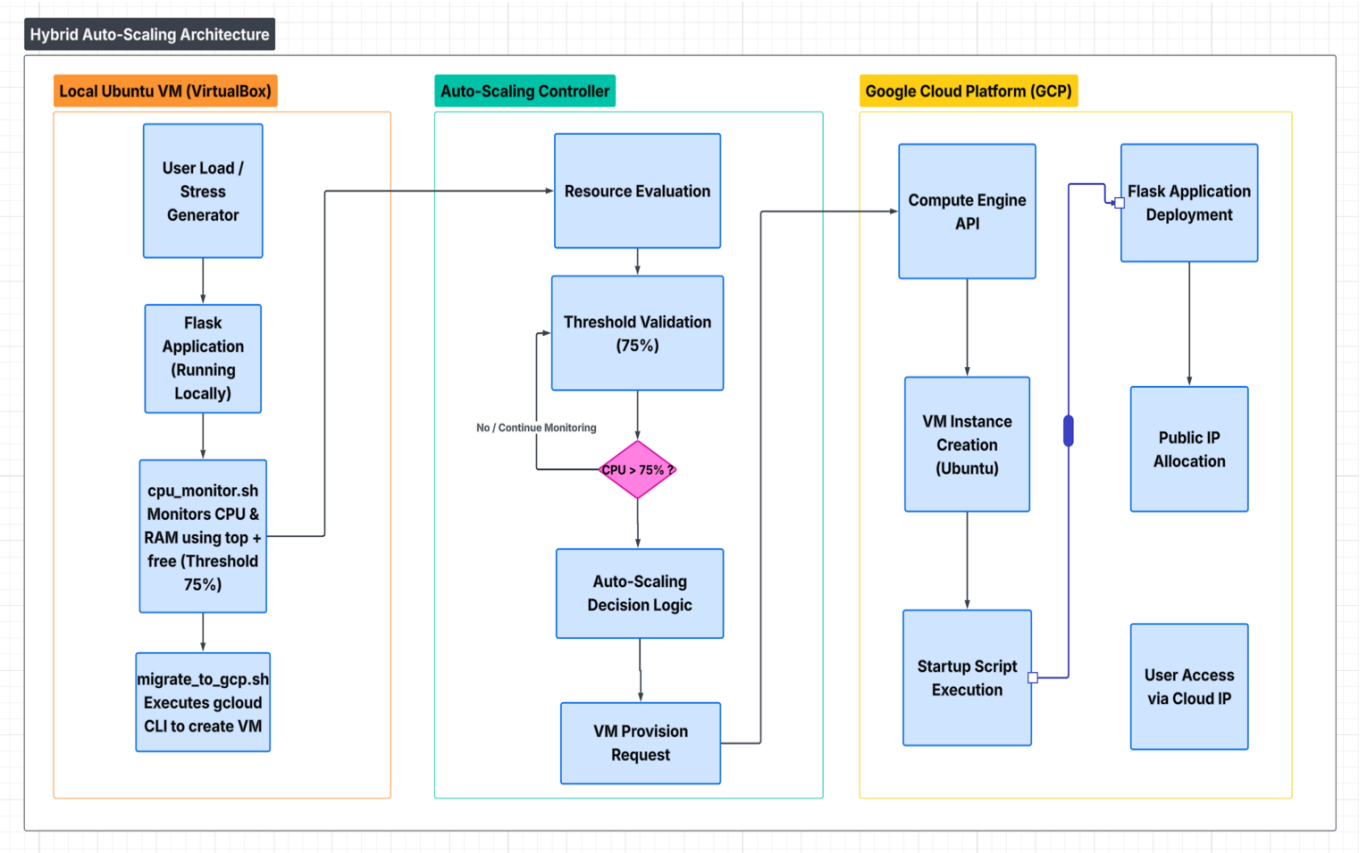
2. Technologies Used

Component	Technology
Local Virtualization	Oracle VM VirtualBox
Local OS	Ubuntu
Cloud Provider	Google Cloud Platform
Cloud CLI	Google Cloud SDK
Backend Monitoring	Flask
Programming Language	Python
Monitoring Tools	stress-ng, htop

3. System Architecture

Flow Explanation:

1. User generates CPU load in Local Ubuntu VM
2. Python monitoring script checks CPU usage
3. If CPU > 75%:
    - gcloud compute instances create
4. Cloud VM auto-scales dynamically
5. If CPU does not exceed threshold:
    - System continues monitoring.



The architecture represents a hybrid cloud model combining on-premise virtualization with public cloud elasticity.

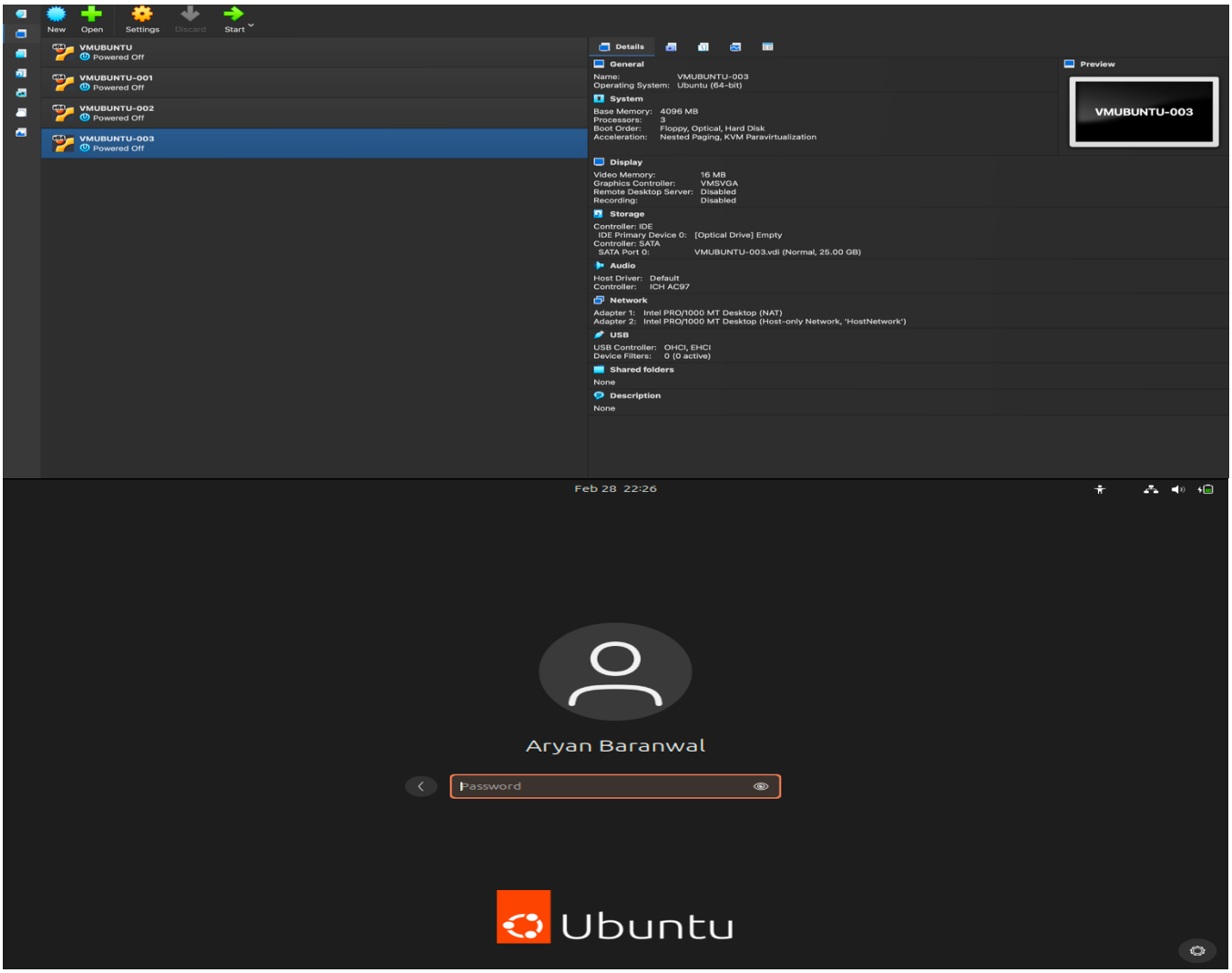
4. Step-by-Step Implementation

4.1 Local Virtual Machine Setup

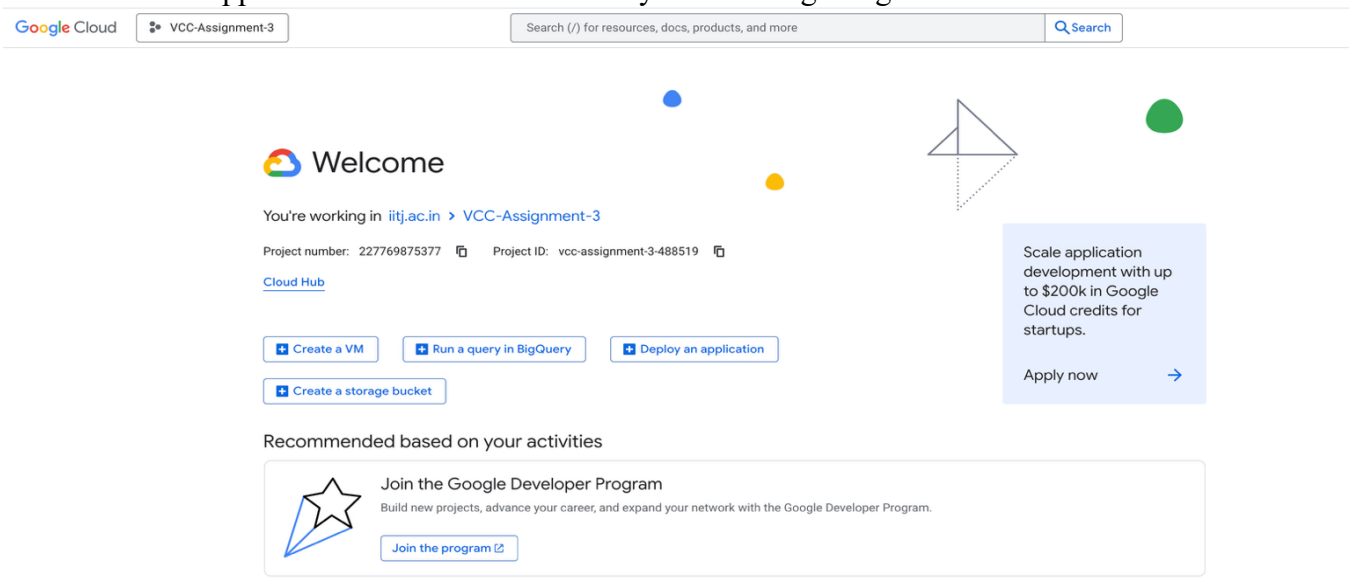
1. Install VirtualBox
2. Installed VirtualBox on host system
3. Open VirtualBox
4. Click **New**
5. Configure:

    - Name: VMUBUNTU-003
    - Type: Linux
    - Version: Ubuntu 24.04 LTS (64-bit)
    - RAM: 4GB
    - CPU: 3 cores
    - Storage: 25GB
4. Attach Ubuntu ISO
5. Start VM
6. Complete Ubuntu installation

Local VM successfully created.



Necessary packages such as curl, git, stress-ng, htop, python3-pip, and Flask were installed.
The local Flask application was tested successfully before integrating with cloud infrastructure.



4.2 Implementation of Resource Monitoring

A custom monitoring script was implemented to track:

- CPU utilization
- RAM usage

The system continuously checks resource usage. If CPU or RAM exceeds 75%, migration to cloud is triggered. Load testing was performed using a stress tool to artificially increase CPU usage and validate the monitoring mechanism.

When load exceeded the threshold:

- Auto-scaling script executed
- Cloud VM creation process started

When load normalized:

- Cloud VM was automatically terminated

This confirms dynamic scale-up and scale-down functionality.

```
(flaskenv) aryanbaranwal@aryanbaranwal-VirtualBox:~/hybrid-gcp$ ./cpu_monitor.sh
CPU=51.5% RAM=60.1381%
SYSTEM NORMAL
(flaskenv) aryanbaranwal@aryanbaranwal-VirtualBox:~/hybrid-gcp$ stress-ng --cpu $(nproc) --cpu-load 90 --timeout 120s
stress-ng: info:  [11637] setting to a 2 mins, 0 secs run per stressor
stress-ng: info:  [11637] dispatching hogs: 3 cpu
stress-ng: info:  [11639] cpu: for stable load results, select a specific cpu stress method with --cpu-method other than
 'all'
stress-ng: info:  [11641] cpu: for stable load results, select a specific cpu stress method with --cpu-method other than
 'all'
stress-ng: info:  [11640] cpu: for stable load results, select a specific cpu stress method with --cpu-method other than
 'all'
stress-ng: info:  [11637] skipped: 0
stress-ng: info:  [11637] passed: 3: cpu (3)
stress-ng: info:  [11637] failed: 0
stress-ng: info:  [11637] metrics untrustworthy: 0
stress-ng: info:  [11637] successful run completed in 2 mins, 0.03 secs
(flaskenv) aryanbaranwal@aryanbaranwal-VirtualBox:~/hybrid-gcp$
```

4.3 Configuration of Cloud Auto-Scaling and Resource Migration

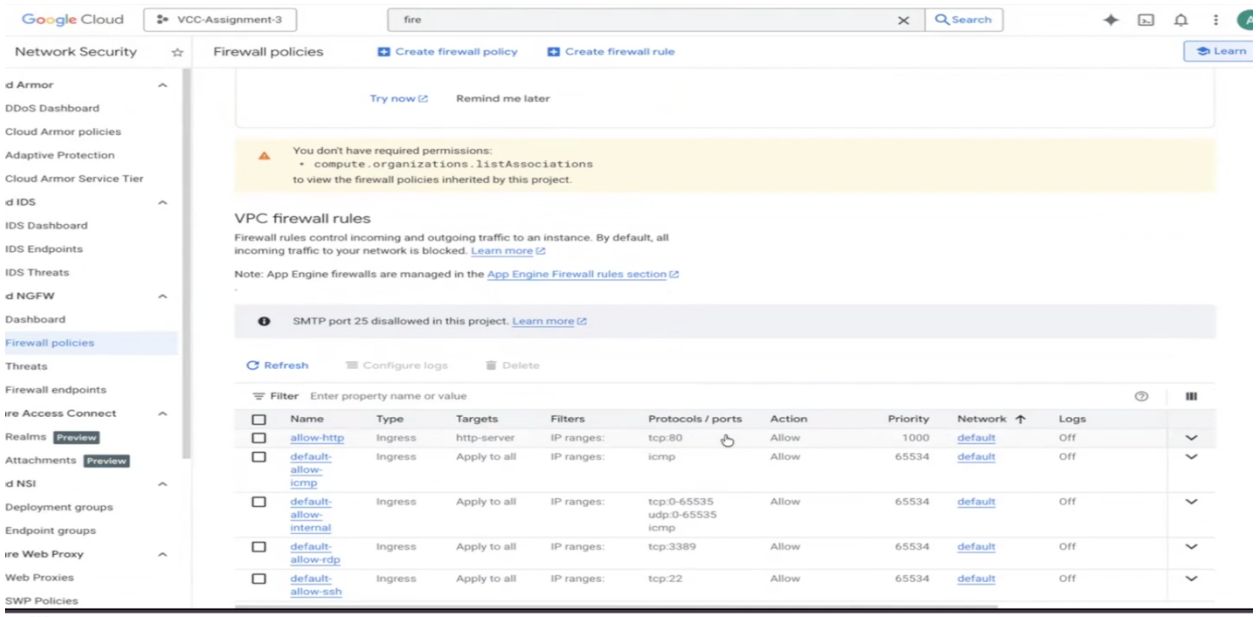
Google Cloud CLI was installed and authenticated within the local Ubuntu VM.

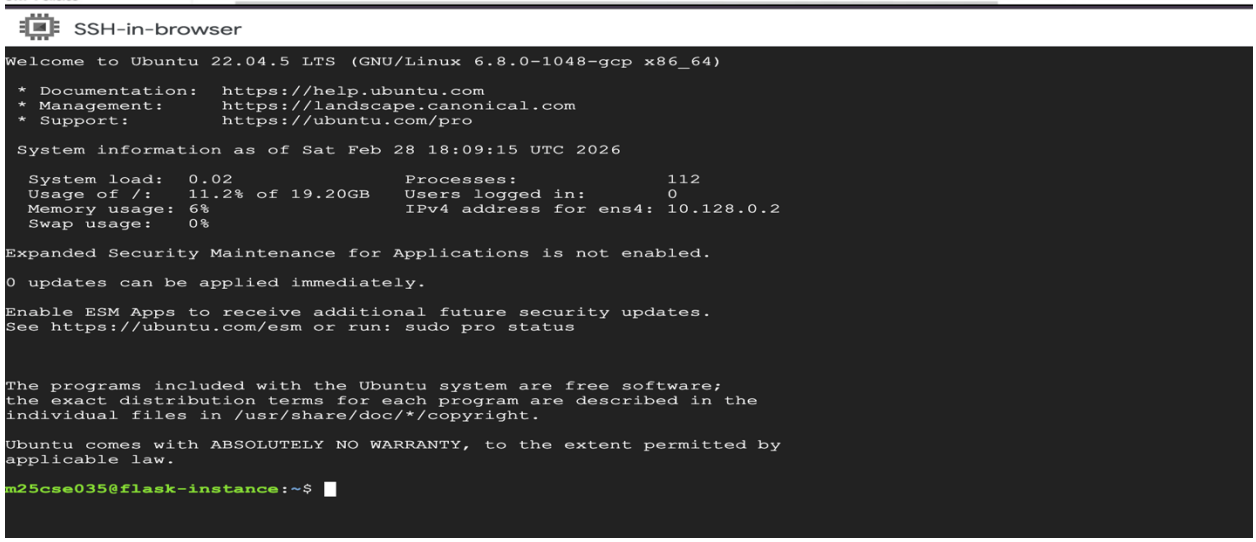
The active project and compute zone were configured to ensure proper deployment.

An automated provisioning script was created to:

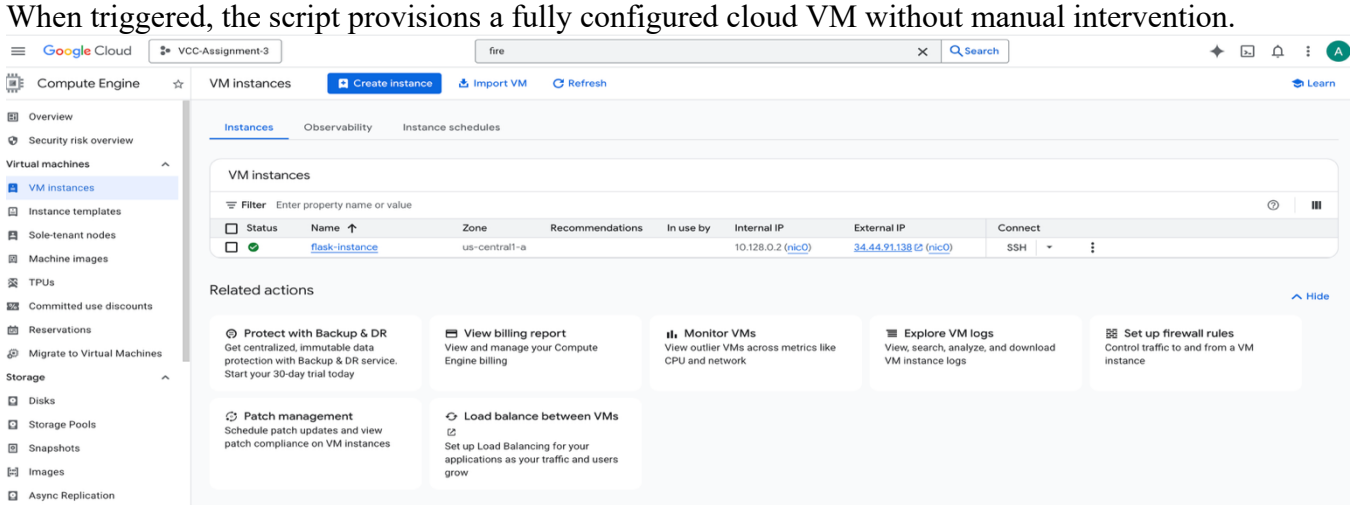
- Create a new cloud VM instance
- Configure machine type (e2-micro)
- Install required dependencies

- Deploy Flask application automatically
- Allow HTTP traffic via firewall rules

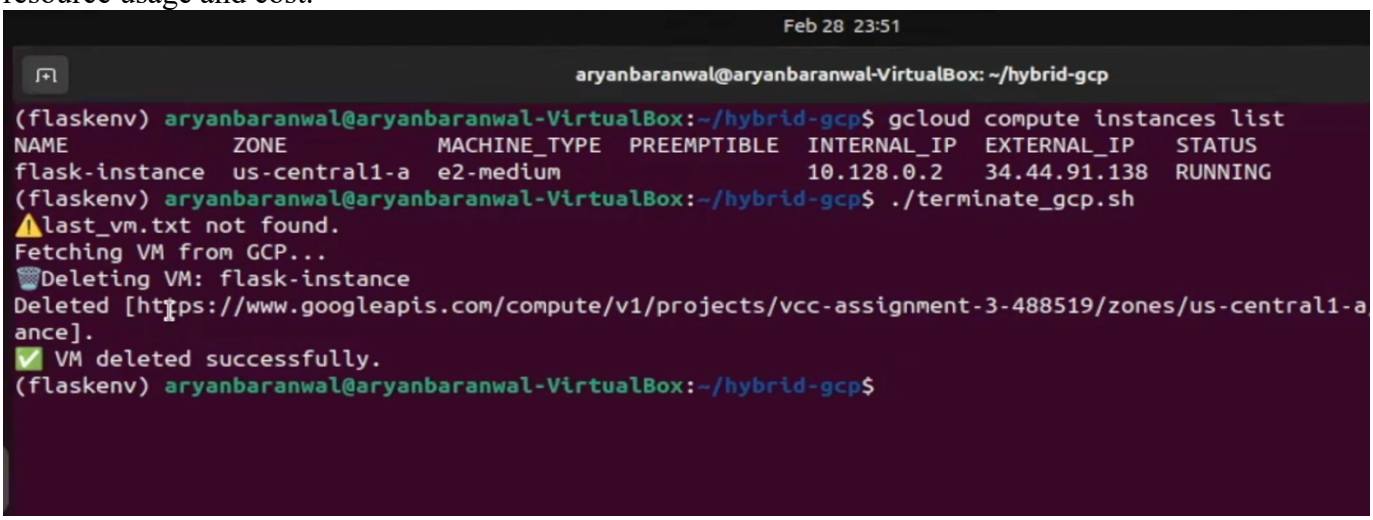




When triggered, the script provisions a fully configured cloud VM without manual intervention.



When load returns to normal levels, a termination script deletes the cloud VM to avoid unnecessary resource usage and cost.



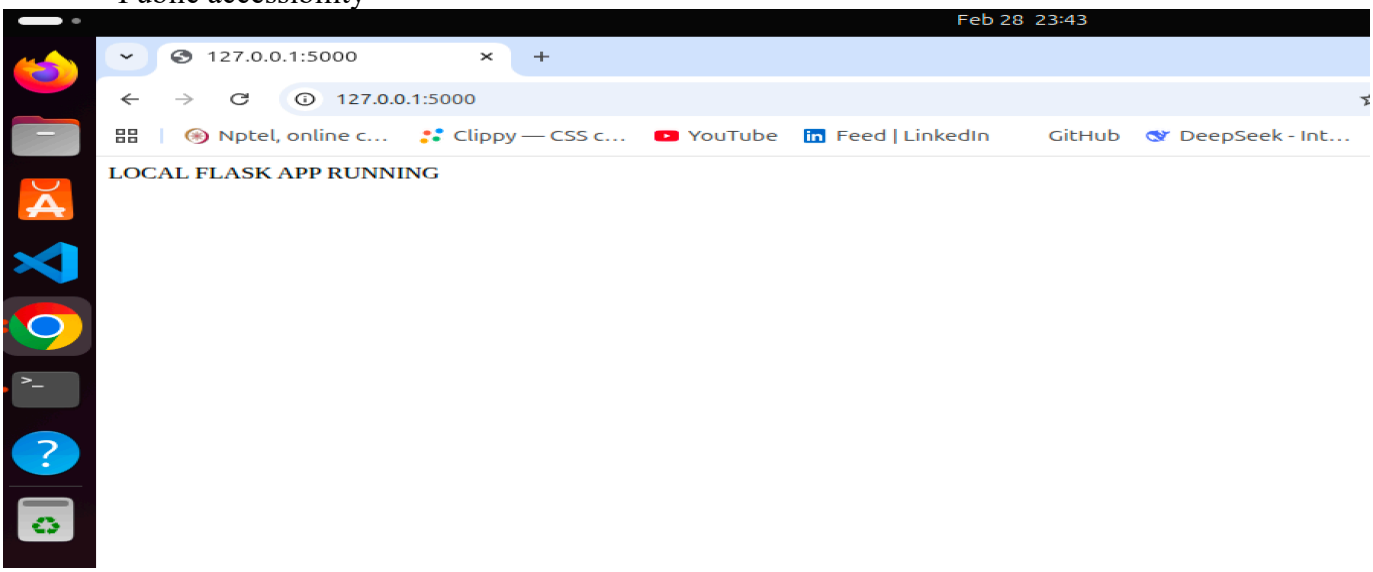
4.4 Deployment of Sample Application

A simple Flask-based web application was developed.
The application was:

- Hosted locally initially
- Automatically deployed to the cloud VM during scaling
- Configured to run on Port 80 for public access

After VM provisioning, the external IP address was accessed through a web browser.
The successful display of the application confirmed:

- Proper VM provisioning
- Successful dependency installation
- Firewall configuration correctness
- Public accessibility

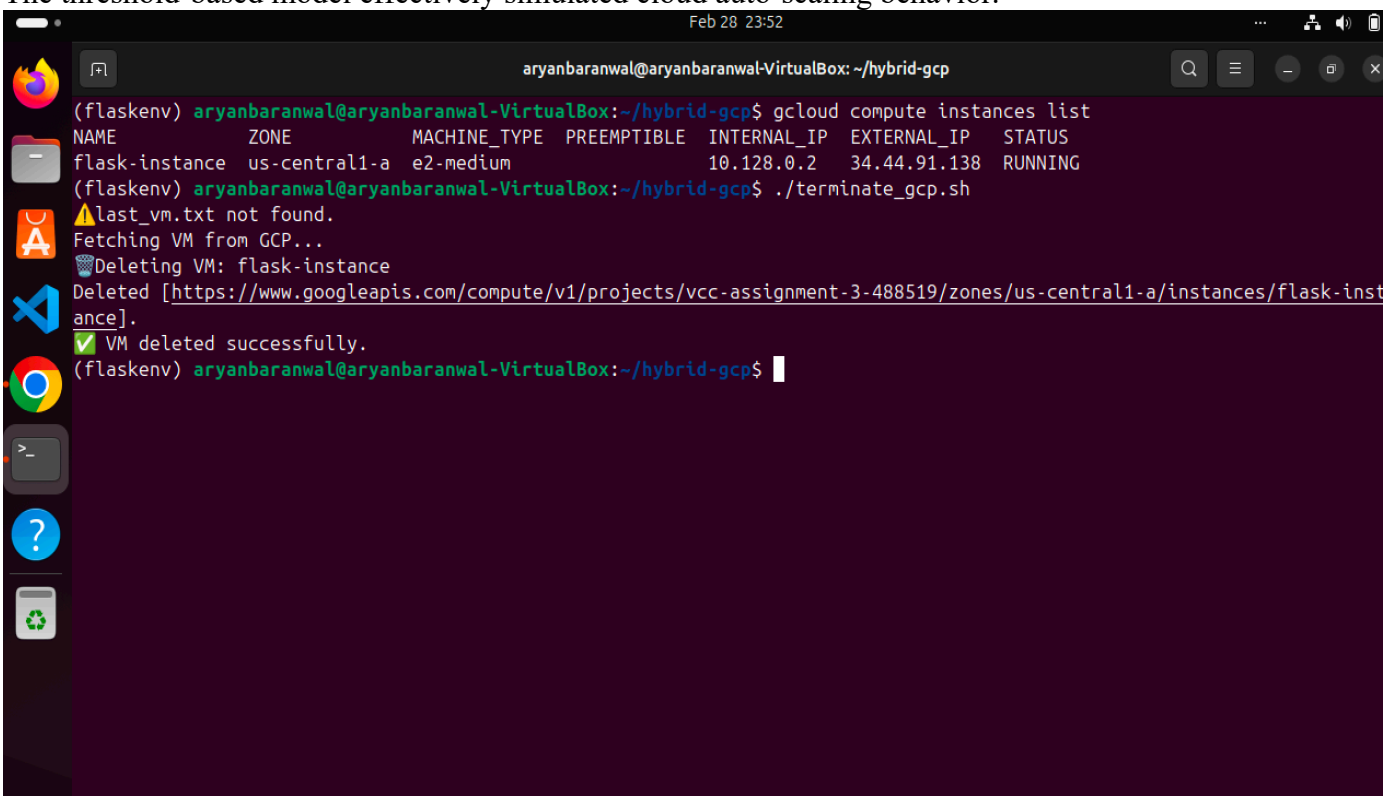


5. RESULTS AND OBSERVATIONS

The system successfully demonstrated:

- Real-time resource monitoring
- Automatic cloud provisioning
- Application deployment automation
- Public accessibility of cloud-hosted service
- Automatic termination during low usage

The threshold-based model effectively simulated cloud auto-scaling behavior.



6. ADVANTAGES

- Cost-efficient resource usage
- Automatic scaling without manual intervention
- Real-world hybrid cloud simulation
- Practical demonstration of Infrastructure as Code
- Improved system scalability

7. LIMITATIONS

- Scaling limited to a single cloud instance
- No load balancer integration
- No container orchestration
- Monitoring is threshold-based rather than predictive

8. FUTURE ENHANCEMENTS

• Integration with Kubernetes for container auto-scaling
• Addition of load balancer for traffic distribution
• Implementation of predictive scaling
• Integration with monitoring dashboards (Prometheus/Grafana)
• Multi-instance scaling support

9. SOURCE CODE REPOSITORY

GitHub Repository Link: https://github.com/aryan9867bar/hybrid-vm-auto-scaling

10. VIDEO DEMONSTRATION LINK

Recorded Video Demo Link:
https://drive.google.com/file/d/1PrMnJCgqFLXQdKyQjKeVxMOBJBIAeB8i/view?usp=sharing

11. CONCLUSION

This assignment successfully demonstrates the implementation of a hybrid virtualization system with threshold-based auto-scaling to Google Cloud Platform.
The project validates practical understanding of:

- Virtualization
- Cloud Integration
- Resource Monitoring
- Infrastructure Automation
- Hybrid Cloud Architecture

The implementation simulates real-world elastic cloud systems and provides hands-on experience in modern cloud infrastructure management.